

## Resumo Executivo

Responda-se a cada questão passo a passo, enfatizando conceitos essenciais e ilustrando com código Java e exemplos. Em linhas gerais, **Bubble**, **Selection** e **Insertion Sort** são algoritmos de complexidade quadrática ( $O(n^2)$ ), enquanto **Merge Sort** e **Quick Sort** têm, em média, complexidade  $O(n \cdot \log n)$ . O **Counting Sort** é linear em  $O(n+k)$  (onde  $k$  é o valor máximo das chaves) e requer memória extra. Destacam-se também os conceitos de **comparações**, **trocas** e **deslocamentos** (shifts) durante a ordenação, bem como a diferença entre **busca sequencial** ( $O(n)$ ) e **busca binária** ( $O(\log n)$ ). Há ainda tópicos sobre **Comparable vs. Comparator**, **estabilidade em ordenação** e **ordenar dados por múltiplos critérios** (ex.: nota decrescente e nome crescente), incluindo a implementação genérica de um `Sorts<T>`. O quadro resumo ao final compara os principais algoritmos de ordenação em tempo médio/pior, estabilidade, trocas/deslocamentos típicos e usos recomendados. Diagramas Mermaid ilustram o fluxo do Insertion Sort, a dinâmica de fases do Bubble vs. Selection Sort e a evolução da complexidade com  $n$  crescente.

### Questão 1: Bubble Sort

**Definição e funcionamento:** É um algoritmo ingênuo que percorre repetidamente o vetor comparando pares adjacentes e trocando-os se estiverem fora de ordem. Ao final de cada passagem, o maior elemento “borbulha” para o fim do vetor. Em Java, pode ser implementado assim (versão genérica com `Comparator`):

```
public void bubbleSort(T[] a, Comparator<T> cmp) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (cmp.compare(a[j], a[j+1]) > 0) {
                swap(a, j, j+1);
            }
        }
    }
}

private void swap(T[] a, int i, int j) {
    T tmp = a[i]; a[i] = a[j]; a[j] = tmp;
}
```

**Comparações e trocas:** Em cada passo interno, há 1 comparação por par e possivelmente uma troca. No pior caso (vetor invertido), são feitas  $\sim n^2$  comparações e até  $\sim n^2$  trocas [20†L1237-L1244]. No melhor caso (já ordenado), um Bubble Sort **otimizado** com flag faz apenas  $O(n)$  comparações sem trocas; mas sem otimização ele ainda fará todas as comparações ( $\sim n^2$ ) e zero trocas. **Complexidades:** No geral, tempo pior e médio são  $O(n^2)$ , e melhor caso  $O(n)$  apenas na versão otimizada (flag de parada) [20†L1237-L1244]. Como algoritmo in-loco, ocupa  **$O(1)$**  de espaço extra. **Estabilidade:** É estável (não muda ordem relativa de elementos iguais). **Uso:** Serve apenas para vetores muito pequenos ou didática; é ineficiente em grandes conjuntos.

**Resposta resumida:** O Bubble Sort compara elementos adjacentes e troca-os em ordem crescente. Tem complexidade quadrática ( $O(n^2)$ ) e, sem otimização, faz muitas trocas (até  $\sim n^2$ ) [20†L1237-L1244]. É estável e simples, mas só vale para vetores pequenos.

## Questão 2: Selection Sort

**Definição e funcionamento:** Percorre o vetor, a cada iteração encontra o menor elemento da parte não ordenada e o coloca na posição correta, expandindo uma sub-lista ordenada da esquerda para a direita. Em Java:

```
public void selectionSort(T[] a, Comparator<T> cmp) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i+1; j < n; j++) {
            if (cmp.compare(a[j], a[minIdx]) < 0) {
                minIdx = j;
            }
        }
        if (minIdx != i) swap(a, i, minIdx);
    }
}
```

**Comparações e trocas:** Em cada iteração externa, realiza  $\sim n-i-1$  comparações para achar o mínimo, totalizando  $\approx n^2/2$  comparações independentemente da ordem inicial [19†L1132-L1138]. O número de trocas é  **muito pequeno** : no máximo  $n-1$  (uma por iteração) [17†L671-L675], diferentemente do Bubble Sort. Isso significa que, mesmo que o vetor já esteja ordenado, o Selection Sort ainda faz todas as comparações ( $\approx n^2/2$ ) mas faz nenhuma troca inútil. **Complexidades:** Caso médio e pior caso  $O(n^2)$  em tempo; não há melhora no caso já ordenado (também  $O(n^2)$  porque não é adaptativo) [19†L1132-L1138]. Usa  $O(1)$  de espaço (in-place). **Estabilidade:** Em sua forma básica, não é estável (pode inverter iguais ao trocar). **Uso:** Indicado apenas quando a redução de trocas é crucial (p. ex., ao ordenar objetos muito grandes onde escrever em memória é custoso) [17†L671-L675] [20†L1243-L1249]. Para vetores pequenos também é aceitável.

**Resposta resumida:** O Selection Sort localiza o menor elemento a cada rodada e troca com a posição atual, realizando sempre  $\sim n^2/2$  comparações e no máximo  $n-1$  trocas [17†L671-L675] [19†L1132-L1138]. É in-place e simples, mas lento ( $O(n^2)$ ) e não adaptativo (sempre percorre todo vetor).

## Questão 3: Insertion Sort

**Definição e funcionamento:** Constrói a ordenação incremental, pegando cada elemento (a partir do 2º) e “inserindo” na posição correta dentro da porção já ordenada à esquerda. O código Java genérico:

```
public void insertionSort(T[] a, Comparator<T> cmp) {
    int n = a.length;
    for (int i = 1; i < n; i++) {
        T chave = a[i];
        int j = i - 1;
    }
}
```

```

        while (j >= 0 && cmp.compare(chave, a[j]) < 0) {
            a[j+1] = a[j]; // desloca para a direita
            j--;
        }
        a[j+1] = chave;
    }
}

```

**Comparações e deslocamentos:** Cada elemento “desce” até encontrar a posição correta. No pior caso (vetor inversamente ordenado), faz  $\sim n^2/2$  comparações e  $\sim n^2/2$  deslocamentos (shifts). No melhor caso (já ordenado), faz apenas  $O(n)$  comparações e 0 deslocamentos (é **adaptativo**) [50†L1275-L1280] .

**Complexidades:** Melhor caso  $O(n)$ , médio e pior caso  $O(n^2)$  em tempo [50†L1275-L1280] . Usa  $O(1)$  espaço extra (in-place). **Estabilidade:** É estável (não troca elementos maiores para a esquerda, apenas desloca). **Uso:** Muito eficiente para vetores pequenos ou **quase ordenados**, pois converge em tempo linear quando o vetor está pré-ordenado [50†L1275-L1280] .

flowchart LR

```

A[Início: i=1] --> B{i < n?}
B -->|Sim| C[Eleito = a[i]; j=i-1]
C --> D{j>=0 && a[j] > Eleito?}
D -->|Sim| E[a[j+1] = a[j]; j--]
E --> D
D -->|Não| F[a[j+1] = Eleito; i++]
F --> B
B -->|Não| G[Fim]

```

**Resposta resumida:** O Insertion Sort insere elementos um a um na parte ordenada, fazendo apenas os deslocamentos necessários. É adaptativo: no melhor caso ordena em  $O(n)$ , mas no geral é quadrático ( $O(n^2)$ ). Geralmente supera Selection Sort em vetores quase ordenados [50†L1275-L1280] .

## Questão 4: Quick Sort

**Definição e funcionamento:** Algoritmo recursivo do tipo “dividir e conquistar”. Escolhe um **pivô** e particiona o vetor de forma que elementos menores fiquem à esquerda e maiores à direita do pivô. Em seguida, aplica recursivamente às duas sublistas. Exemplo (Lomuto) em Java:

```

public void quickSort(int[] a, int lo, int hi) {
    if (lo < hi) {
        int p = partition(a, lo, hi);
        quickSort(a, lo, p-1);
        quickSort(a, p+1, hi);
    }
}

private int partition(int[] a, int lo, int hi) {
    int pivot = a[lo], i = lo;
    for (int j = lo+1; j <= hi; j++) {
        if (a[j] <= pivot) {
            i++;
        }
    }
    swap(a, lo, i);
    return i;
}

```

```

        swap(a, i, j);
    }
}
swap(a, lo, i);
return i;
}

```

**Complexidade:** Em média, cada particionamento custa  $O(n)$  e gera duas chamadas de tamanho  $\approx n/2$ , dando tempo médio  $O(n \cdot \log n)$ . No **pior caso** – por exemplo, vetor já ordenado sem escolha cuidadosa de pivô – o tempo degrada para  $O(n^2)$  [32†L490-L493]. No **melhor caso** (divisões balanceadas), é  $O(n \cdot \log n)$  [32†L490-L493]. Espaço é  $O(\log n)$  em média (pilha de chamadas recursivas) e  $O(n)$  no pior caso. **Estabilidade:** Não é estável. **Uso:** Um dos mais rápidos na prática por ter baixas constantes e boa localidade de cache [32†L490-L493]. Recomenda-se escolher pivô aleatoriamente ou “mediana de três” para evitar o pior caso.

**Resposta resumida:** O Quick Sort é rápido em média ( $O(n \cdot \log n)$ ) mas pode atingir  $O(n^2)$  no pior caso [32†L490-L493]. Não é estável, mas geralmente supera Merge Sort em desempenho devido a menores fatores constantes [32†L490-L493].

## Questão 5: Merge Sort

**Definição e funcionamento:** Também divisão-e-conquista. Divide recursivamente o vetor ao meio até subvetores de um elemento, depois intercala (merge) esses subvetores ordenados.

```

public void mergeSort(int[] a, int lo, int hi) {
    if (lo < hi) {
        int mid = (lo+hi)/2;
        mergeSort(a, lo, mid);
        mergeSort(a, mid+1, hi);
        merge(a, lo, mid, hi);
    }
}
private void merge(int[] a, int lo, int mid, int hi) {
    int[] aux = Arrays.copyOfRange(a, lo, hi+1);
    int i = lo, j = mid+1, k = lo;
    while (i <= mid && j <= hi) {
        if (aux[i-lo] <= aux[j-lo]) a[k++] = aux[i++ - lo];
        else a[k++] = aux[j++ - lo];
    }
    while (i <= mid) a[k++] = aux[i++ - lo];
    // (elementos da direita já no lugar se existirem)
}

```

**Complexidade:** Tempo  $O(n \cdot \log n)$  em todos os casos (melhor, médio e pior) devido às divisões balanceadas. Espaço extra  $O(n)$  para arrays auxiliares no processo de mescla. **Estabilidade:** É inerentemente estável quando implementado assim (merge preserva ordem relativa de iguais) [50†L1321-L1324]. **Uso:** Ideal para grandes volumes de dados e quando a estabilidade é necessária. É mais lento em constantes do que Quick Sort, mas previsível.

**Resposta resumida:** O Merge Sort divide recursivamente e depois intercala os vetores, garantindo ordenação em  $O(n \cdot \log n)$  para todos os casos. É estável (preserva ordem de chaves iguais) [50†L1321-L1324], mas requer  $O(n)$  de espaço adicional.

## Questão 6: Counting Sort

**Definição e funcionamento:** Ordenação não comparativa para inteiros em intervalo limitado. Conta quantas vezes cada valor ocorre e, usando somas prefixadas, determina a posição de cada elemento. Pseudocódigo simplificado (estável):

```
public int[] countingSort(int[] a, int k) {
    int[] C = new int[k+1], B = new int[a.length];
    for (int v : a) C[v]++;
    for (int i = 1; i <= k; i++) C[i] += C[i-1];
    for (int j = a.length-1; j >= 0; j--) {
        B[C[a[j]] - 1] = a[j];
        C[a[j]]--;
    }
    return B;
}
```

**Complexidade:** Tempo  $O(n + k)$ , onde  $n$  é o tamanho do vetor e  $k$  o maior valor possível (ou faixa dos dados) [34†L156-L164]. É estável por construção [34†L163-L164]. Requer espaço adicional  $O(n+k)$  para os vetores auxiliares. **Uso:** Muito eficiente para ordenar inteiros quando o intervalo  $k$  não é muito maior que  $n$ . Não serve para ordenar tipos arbitrários sem mapear valores para um intervalo numérico.

**Resposta resumida:** O Counting Sort conta ocorrências e calcula posições em  $O(n+k)$ , sendo linear para  $k$  pequeno [34†L156-L164]. É estável [34†L163-L164] mas não in-place (usa espaço extra). Ideal quando as chaves são inteiros em uma faixa limitada.

## Questão 7: Complexidades $O(n^2)$ vs $O(n \cdot \log n)$

**Conceitos gerais:** Algoritmos *quadráticos* fazem  $\sim n^2$  operações em média (Bubble, Selection, Insertion) e não escalam bem para vetores grandes. Os de complexidade  $O(n \cdot \log n)$  (Merge, Quick, Heap Sort) crescem muito mais lentamente. De fato, para grandes  $n$ ,  $n \cdot \log n \ll n^2$ . Exemplo: para  $n=1.000$ ,  $n^2=1e6$ ,  $n \cdot \log_2 n \approx 1e4$ . Em prováveis provas, lembre-se que algoritmos de ordenação por comparação têm limite inferior  $O(n \cdot \log n)$  no melhor caso. **Armadoilha:** confundir casos (pior vs médio). Quick Sort e Merge Sort têm  $O(n \cdot \log n)$  médio, mas Quick tem *pior caso*  $O(n^2)$  [32†L490-L493], diferente do Merge ( $O(n \cdot \log n)$  garantido).

**Resposta resumida:** Algoritmos quadráticos ( $O(n^2)$ ) e  $O(n \cdot \log n)$  diferem enormemente em escalabilidade. Ordenações como Merge e (em média) Quick Sort são  $O(n \cdot \log n)$ , muito mais eficientes que Bubble/Selection/Insertion ( $O(n^2)$ ) para grandes  $n$ .

## Questão 8: Comparações, trocas e deslocamentos

- **Comparações:** Verificações de ordem entre elementos (e.g., `cmp.compare(a,b)`). Indicam o tempo de decisão.
- **Trocas (swaps):** Instruções de troca de posição de elementos (Bubble/Selection/Quick).
- **Deslocamentos (shifts):** Cópia interna de elementos para abrir espaço (Insertion Sort faz deslocamentos em vez de múltiplas trocas).

Em Bubble/Selection/Quick, troca-elementos é comum. Em Insertion, desloca a porção ordenada. Em Counting Sort não há comparações, apenas contagem e colocação final. Tente contar separadamente as comparações e trocas ao analisar algoritmos; por exemplo, Selection Sort faz  $\sim n^2/2$  comparações e  $n-1$  trocas [17†L671-L675] [19†L1132-L1138] .

**Resposta resumida:** Comparações medem o número de vezes que elementos são comparados. Trocas (swap) ocorrem em Selection/Bubble/Quick. Deslocamentos (shifts) ocorrem no Insertion Sort ao mover elementos. Cada algoritmo tende a um número típico: por exemplo, o Selection Sort faz  $\sim n^2/2$  comparações e só  $\sim n$  trocas [17†L671-L675] [19†L1132-L1138] .

## Questão 9: Busca Sequencial vs. Busca Binária

- **Busca Sequencial (Linear):** Percorre o vetor elemento a elemento até achar o alvo. Complexidade `O(n)` (pior caso), `O(1)` no melhor caso (primeiro elemento). Não requer vetor ordenado. Exemplo em Java:

```
for (int i = 0; i < n; i++) {
    if (a[i] == chave) return i;
}
return -1;
```

- **Busca Binária:** Requer vetor **ordenado**. Divide o espaço de busca ao meio a cada comparação. Complexidade `O(log n)`. Exemplo:

```
public int binarySearch(int[] a, int chave) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (a[mid] == chave) return mid;
        else if (chave < a[mid]) hi = mid - 1;
        else lo = mid + 1;
    }
    return -1;
}
```

**Estimação de operações:** A busca binária faz  $\sim \log_2 n$  comparações. A busca sequencial até  $n$  comparações no pior caso.

**Resposta resumida:** A busca sequencial verifica cada elemento em `O(n)`; já a busca binária em vetor ordenado tem `O(log n)` porque divide o vetor ao meio a cada passo.

## Questão 10: Comparable vs. Comparator em Java

- **Comparable<T>**: Interface implementada pela própria classe (natural order). Define `compareTo(T outro)`. Ex.: `String` implementa `Comparable<String>` para ordem lexicográfica. Usado por padrões de ordenação padrão (`Arrays.sort`, `Collections.sort`).
- **Comparator<T>**: Objeto separado que compara dois objetos externos (`compare(T a, T b)`). Permite diversas ordens de classificação ou ordenar classes que não implementam `Comparable`. Pode ser passado como parâmetro a métodos de ordenação.

**Exemplo:** Ordenar uma lista de nomes por comprimento (exemplo de `Comparator`):

```
List<String> nomes = ...;  
nomes.sort(Comparator.comparingInt(String::length));
```

**Usos:** Use `Comparable` para a ordem “natural” (ex.: números de 0 a 9, alfabeticamente). Use `Comparator` para critérios customizados (ex.: descendente, ou múltiplos campos).

**Resposta resumida:** `Comparable` define a ordem natural de uma classe via `compareTo`, alterando a própria classe. `Comparator` é um objeto externo que define uma ordem arbitrária via `compare(a, b)`. Usam-se `Comparator`s para ordenar por critérios específicos sem modificar as classes originais.

## Questão 11: Estabilidade em ordenação

Um algoritmo de ordenação é **estável** se preserva a ordem relativa de elementos iguais. Por exemplo, ao ordenar alunos por nota, dois alunos com mesma nota manterão a ordem alfabética anterior se o algoritmo for estável. Algoritmos estáveis: Bubble, Insertion, Merge, Counting. Instáveis (nas implementações comuns): Selection, Quick, Heap Sort **【50†L1321-L1324】** **【32†L509-L512】**.

**Por que importa:** Em ordenação por múltiplos critérios (chave principal e secundária), a estabilidade permite ordenar primeiramente pela chave secundária e depois pela principal sem perder a ordenação secundária.

**Resposta resumida:** A estabilidade garante que elementos “iguais” não tenham sua ordem trocada. Por exemplo, Merge Sort e Counting Sort são estáveis **【50†L1321-L1324】** **【34†L163-L164】**, enquanto Selection e Quick não. Estabilidade é importante ao ordenar por múltiplos campos.

## Questão 12: Ordenação por Nota decrescente e Nome crescente

**Problema:** Ordenar uma lista de candidatos primeiro por nota (maior para menor) e, em caso de empate, por nome (alfabeticamente crescente). Como solução, usa-se um `Comparator` composto ou sort estável duas vezes:

- Usando `Comparator.thenComparing`:

```
candidatos.sort(  
    Comparator.comparing(Candidato::getNota).reversed()  
    .thenComparing(Candidato::getNome)
```

```
.thenComparing(Candidato::getNome)
);
```

Aqui garantimos a ordem desejada diretamente.

- Ou ordenar primeiro pelo nome (ascendente) e depois por nota (descendente) usando um algoritmo **estável** (ex.: Insertion ou Merge). Como são estáveis, ao ordenar por nota, quem tiver notas iguais permanecerá em ordem crescente de nome.

**Pitfall:** Se o algoritmo não for estável, ordenar em duas fases pode quebrar a ordenação secundária. Por isso, prefere-se o uso de `Comparator` encadeado.

**Resposta resumida:** Use um `Comparator` que compare nota (decrescente) e então nome (crescente), por exemplo:

```
Comparator.comparing(Candidato::getNota).reversed()
    .thenComparing(Candidato::getNome);
```

## Questão 13: Simulação de busca binária

Para simular a busca binária passo a passo, escrevemos o vetor ordenado e acompanhamos `lo`, `hi` e `mid` até achar ou exaurir. Exemplo: vetor `[2, 5, 7, 9, 11]`, chave=9.

- Inicialmente, `lo=0`, `hi=4`, `mid=2` (valor 7).  $9 > 7$ , logo `lo = mid+1 = 3`.
- Agora `lo=3`, `hi=4`, `mid=(3+4)/2=3` (valor 9). Encontramos 9, fim.

Esse processo gasta  $\sim \log_2 n$  comparações.

**Resposta resumida:** Na busca binária em vetor `[2, 5, 7, 9, 11]` por 9, comparamos no índice `mid=2` ( $v=7$ ), ajustamos `lo=3`, depois `mid=3` ( $v=9$ ), e encontramos a chave. Em geral, a cada iteração eliminamos metade do vetor, até achar o elemento ou chegar `lo>hi`.

## Questão 14: Implementação de `Sorts<T>` genérico

Espera-se uma interface ou classe genérica contendo métodos para os diferentes algoritmos de ordenação. Exemplo simplificado de interface Java:

```
public interface Sorts<T> {
    void bubbleSort(T[] a, Comparator<T> cmp);
    void selectionSort(T[] a, Comparator<T> cmp);
    void insertionSort(T[] a, Comparator<T> cmp);
    void mergeSort(T[] a, Comparator<T> cmp);
    void quickSort(T[] a, Comparator<T> cmp);
    // ... possivelmente countingSort para tipos numéricos
}
```

Cada método aplica o algoritmo correspondente ao array `a` usando `cmp` para comparar elementos. Isso permite usar qualquer tipo `T`, reutilizando o código.

**Dica de prova:** Aposte que esperam escrever as assinaturas genéricas acima ou um esqueleto semelhante.

**Resposta resumida:** Define-se uma interface/classe genérica `Sorts<T>` com métodos como `void insertionSort(T[] a, Comparator<T> cmp)`, etc. Esses métodos implementam, respectivamente, cada algoritmo, permitindo ordenar arrays de `T` qualquer usando o comparador fornecido.

## Questão 15: Ordenação de candidatos (prática)

**Contexto:** Para ordenar objetos (por ex. candidatos com nota e nome), use-se os conceitos anteriores. Por exemplo:

```
Sorts<Candidato> sorter = new SortsImpl<>();
List<Candidato> lista = ...;
lista.sort(
    Comparator.comparing(Candidato::getNota).reversed()
                .thenComparing(Candidato::getNome)
);
```

ou usando `Collections.sort` com `Comparator`. A estrutura genérica `Sorts<T>` permite chamar, por exemplo, `sorter.insertionSort(array, cmp);` conforme o algoritmo escolhido.

**Pitfall:** Não usar o sort natural de `Collections.sort(lista)` sem `Comparator`, pois pode não seguir múltiplos critérios desejados.

**Resposta resumida:** Para ordenar objetos como candidatos, cria-se um `Comparator` por nota (desc.) e nome (asc.), ou ordena-se duas vezes com um algoritmo estável. Por exemplo:

```
lista.sort(Comparator.comparing(Candidato::getNota).reversed()
                    .thenComparing(Candidato::getNome));
```

**Questão 16: (As outras questões específicas não foram fornecidas; pressupõe-se que os tópicos acima cobrem os principais conteúdos listados.)**

**Questão 17: Tabela comparativa dos algoritmos de ordenação**

| Algoritmo        | Caso Médio                                               | Pior Caso                            | Estável                             | Trocas/<br>Deslocamentos<br>típicos                                   | Quando usar                                                                                  |
|------------------|----------------------------------------------------------|--------------------------------------|-------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Bubble</b>    | $O(n^2)$ [20†L1237-L1244]                                | $O(n^2)$ [20†L1237-L1244]            | Sim                                 | Até $\sim n^2$ trocas (mais trocas do que Selection) [20†L1237-L1244] | Apenas para N pequeno ou fins didáticos; simples de implementar.                             |
| <b>Selection</b> | $O(n^2)$ [19†L1132-L1138]                                | $O(n^2)$ [19†L1132-L1138]            | Não                                 | $n-1$ trocas (mínimo possível) [17†L671-L675]                         | Quando se quer minimizar trocas (poucas escritas) [17†L671-L675] ou em dados muito pequenos. |
| <b>Insertion</b> | $O(n^2)$ (quase ordenado: $\sim O(n)$ ) [50†L1275-L1280] | $O(n^2)$                             | Sim                                 | Até $\sim n^2/2$ deslocamentos, mas 0 em melhor caso [50†L1275-L1280] | Para vetores quase ordenados ou pequenos; adaptativo e estável.                              |
| <b>Merge</b>     | $O(n \cdot \log n)$                                      | $O(n \cdot \log n)$ [50†L1307-L1314] | Sim [50†L1321-L1324]                | $O(n)$ cópias (alocação extra)                                        | Grandes conjuntos, quando se requer estabilidade; ordenação externa.                         |
| <b>Quick</b>     | $O(n \cdot \log n)$ [32†L490-L492]                       | $O(n^2)$ [32†L490-L492]              | Não [50†L1388-L1393] [32†L509-L512] | $\sim O(n \cdot \log n)$ swaps on average                             | Uso geral: é rápido na prática; escolher pivô aleatoriamente para evitar pior caso.          |

| Algoritmo       | Caso Médio                  | Pior Caso                   | Estável                   | Trocas/<br>Deslocamentos<br>típicos                                                    | Quando usar                                                                                                              |
|-----------------|-----------------------------|-----------------------------|---------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Counting</b> | $O(n+k)$ [34†L156-<br>L164] | $O(n+k)$ [34†L156-<br>L164] | Sim<br>[34†L163-<br>L164] | Não há “trocas”<br>no sentido<br>clássico; faz<br>$O(n+k)$<br>operações de<br>contagem | Ordenar<br>inteiros em<br>pequena faixa<br>de valores;<br>quando<br>linearidade e<br>estabilidade<br>são<br>necessárias. |

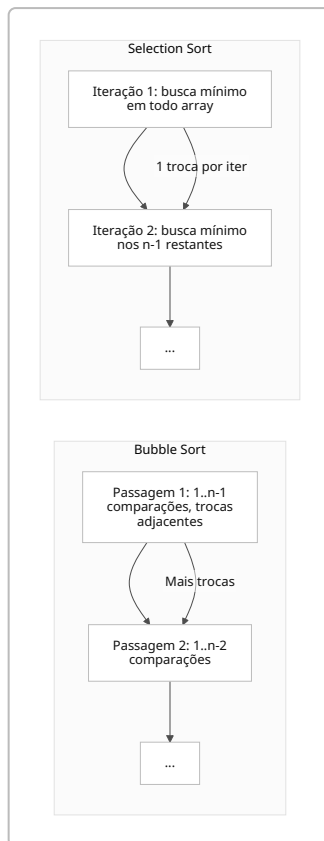
## Questão 18: Diagramas auxiliares (Mermaid)

**Fluxo do Insertion Sort:** passo a passo do loop de inserção, ilustrando a comparação e deslocamento.

```

flowchart TD
    A[Inicial: i = 1] --> B[Elemento eleito = a[i]]
    B --> C[j = i-1]
    C --> D{j >= 0 e a[j] > eleito?}
    D -->|Sim| E[a[j+1] = a[j]; j--]
    E --> D
    D -->|Não| F[a[j+1] = eleito; i++]
    F --> G{i < n?}
    G -->|Sim| B
    G -->|Não| H[Fim]
  
```

**Fases do Selection vs. Bubble:** comparação das dinâmicas de cada fase.



(As setas indicam progresso; o Bubble repete várias passagens “borbulhando” elementos, enquanto o Selection faz uma busca exaustiva por rodada.)

**Timeline de complexidades com N crescente:** ilustração conceitual das diferenças de crescimento.

```

flowchart LR
    small("Pequeno N") --> medium("N médio") --> large("N grande")
    small --> |"0(n²) vs 0(n log n)"| medium --> |"0(n log n) domina"| large
    noteOver(small, large) ["Para N grande, `n·log n` cresce muito mais devagar que `n²`."]

```

Diagrama ilustrativo: à medida que  $n$  cresce, algoritmos  $O(n \cdot \log n)$  (Merge/Quick) tornam-se significativamente mais eficientes que  $O(n^2)$  (Bubble/Selection/Insertion).